

Изменение DataFrames

На предыдущем уроке вы узнали, что такое DataFrame и как выбирать подмножества данных из одного.

В этом уроке вы узнаете, как изменить существующий DataFrame. Вот некоторые из навыков, которые вам предстоит освоить:

- Добавление столбцов в DataFrame
- Использование лямбда-функций для вычисления комплексных величин
- Переименование столбцов

Изменение DataFrames в Pandas

И снова мы будем работать с данными ShoeFly.com, вымышленного интернет-магазина обуви.

```
In [1]: # Before we analyze anything, we need to import pandas
# and load our data
import pandas as pd

df = pd.read_csv('shoefly_messy_orders.csv')
```

Рассмотрим первые 10 строк:

```
In [2]: df.head(10)
```

Out[2]:

	id	first_name	last_name	email	shoe_type	shoe_material
0	74970	Albert	Crane	ACrane1998@gmail.com	ballet flats	fabric
1	28140	Lauren	Wise	LWise1972@gmail.com	Sandles	fabric
2	16841	Bryan	Maldonado	Bryan.Maldonado@gmail.com	STILLETOS	fabric
3	19695	Nancy	Talley	NTalley1988@gmail.com	wedges	fabric
4	61287	Catherine	Brown	CBrown1981@gmail.com	Boots	faux-leather
5	57141	Doris	Newton	DN6902@gmail.com	Sandles	fabric
6	52132	Billy	Mcintyre	BM6854@hotmail.com	wedges	faux-leather
7	25486	Sarah	Kaufman	SK8490@gmail.com	Sandles	leather
8	12421	Susan	Leblanc	SusanLeblanc40@outlook.com	CLOGS	fabric
9	36234	Benjamin	Newton	BenjaminNewton59@aol.com	CLOGS	faux-leather

О, нет! Похоже, наши данные были немного небрежными. Некоторые типы обуви имеют заглавные буквы, а другие - строчные. Это беспорядок, и мы хотели бы его убрать. Мы можем сделать это, применив `string.lower` к столбцу `shoe_type`.

```
In [3]: import string

df['shoe_type'] = df.shoe_type.apply(string.lower)

df.head(10)
```

Намного лучше! Теперь все поля в нижнем регистре.

Наша фабрика сообщает, что больше не может хранить тканевую обувь. Давайте добавим столбец `in_stock`, который имеет значение `True` для всей обуви из нетканой ткани и `False` для обуви из ткани.

```
In [4]: df['in_stock'] = df.shoe_material.apply(lambda x: False if x == 'fabric' else True)

df.head(10)
```

Наш отдел маркетинга хочет объявить о некоторых покупках в нашей ленте Twitter.

Давайте добавим описание к каждой строке, которую они могут использовать. Он появится в новом столбце под названием «описание».

```
In [5]: df['description'] = df.apply(lambda row: "{} {} bought {} {} {}".format(
    row.first_name,
    row.last_name,
    row.shoe_color,
    row.shoe_material,
    row.shoe_type),
    axis=1)

df.head(10)
```

Добавление столбца I

Иногда мы хотим добавить столбец в существующий DataFrame. Возможно, мы захотим добавить новую информацию или выполнить расчет на основе данных, которые у нас уже есть.

Один из способов добавить новый столбец - предоставить список той же длины, что и существующий DataFrame.

Предположим, у нас есть хозяйственный магазин под названием The Handy Woman и есть DataFrame, содержащий информацию об инвентаризации:

Product ID	Product Description	Cost to Manufacture	Price
1	3 inch screw	0.50	0.75
2	2 inch nail	0.10	0.25

Product ID	Product Description	Cost to Manufacture	Price
3	hammer	3.00	5.50
4	screwdriver	2.50	3.00

Похоже, что фактическое количество каждого товара на нашем складе отсутствует!

Давайте воспользуемся следующим кодом, чтобы добавить эту информацию в наш DataFrame.

```
df['Quantity'] = [100, 150, 50, 35]
```

Наш новый DataFrame выглядит так:

Product ID	Product Description	Cost to Manufacture	Price	Quantity
1	3 inch screw	0.50	0.75	100
2	2 inch nail	0.10	0.25	150
3	hammer	3.00	5.50	50
4	screwdriver	2.50	3.00	35

Задание

1. Создайте DataFrame df, который содержит информацию о товарах, продаваемых в хозяйственном магазине. Добавьте в df столбец «Продано оптом?», который указывает, продается ли продукт оптом или по отдельности. Итоговая таблица должна выглядеть так:

Product ID	Product Description	Cost to Manufacture	Price	Sold in Bulk?
1	3 inch screw	0.50	0.75	Yes
2	2 inch nail	0.10	0.25	Yes
3	hammer	3.00	5.50	No
4	screwdriver	2.50	3.00	No

Добавление столбца II

Мы также можем добавить новый столбец, который будет одинаковым для всех строк в DataFrame. Вернемся к нашему примеру с инвентарем:

Product ID	Product Description	Cost to Manufacture	Price
1	3 inch screw	0.50	0.75
2	2 inch nail	0.10	0.25
3	hammer	3.00	5.50
4	screwdriver	2.50	3.00

Предположим, мы знаем, что все наши товары в настоящее время есть в наличии. Мы можем добавить столбец, в котором говорится следующее:

```
df['In Stock?'] = True
```

Теперь во всех строках есть столбец "В наличии?" со значением True.

Product ID	Product Description	Cost to Manufacture	Price	In Stock?
1	3 inch screw	0.50	0.75	True
2	2 inch nail	0.10	0.25	True
3	hammer	3.00	5.50	True
4	screwdriver	2.50		

Задание

1. Добавьте столбец в df под названием «Облагается налогом?», В котором указано, нужно ли собирать налог с продаж на продукт. Для всех строк должно быть указано «Да».

Добавление столбца III

Наконец, вы можете добавить новый столбец, выполнив функцию для существующих столбцов.

Возможно, мы хотим добавить столбец в нашу таблицу инвентаря с суммой налога с продаж, который нам нужно взимать за каждый товар. Следующий код умножает каждую цену на 0,075, налог с продаж для нашего штата:

```
df['Sales Tax'] = df.Price * 0.075
```

Теперь в нашей таблице есть столбец под названием «Налог с продаж»:

Product ID	Product Description	Cost to Manufacture	Price	Sales Tax
1	3 inch screw	0.50	0.75	0.06
2	2 inch nail	0.10	0.25	0.02
3	hammer	3.00	5.50	0.41
4	screwdriver	2.50	3.00	0.22

Задание

1. Добавьте в df (Предыдущая таблица, в которой добавляли налог с продаж) столбец «'Margin'», который равен разнице между ценой и затратами на производство.

Выполнение операций с колонками

В предыдущем упражнении мы узнали, как добавлять столбцы в DataFrame.

Часто столбец, который мы хотим добавить, связан с существующими столбцами, но требует более сложных вычислений, чем умножение или сложение.

Например, представьте, что у нас есть следующая таблица клиентов.

Name	Email
JOHN SMITH	john.smith@gmail.com
Jane Doe	jdoe@yahoo.com
joe schmo	joeschmo@hotmail.com

Немного раздражает, что в каждой строке используются разные заглавные буквы. Возможно, мы хотели бы сделать его более последовательным, сделав все буквы прописными.

Мы можем использовать функцию `apply`, чтобы применить функцию к каждому значению в определенном столбце. Например, этот код перезаписывает существующие столбцы «Имя», применяя функцию `upper` к каждой строке в «Имя».

```
df['Name'] = df.Name.apply(str.upper)
```

Результат:

Name	Email
JOHN SMITH	john.smith@gmail.com
JANE DOE	jdoe@yahoo.com
JOE SCHMO	joeschmo@hotmail.com

Задание

1. Примените функцию ниже ко всем именам в столбце «Имя» в `df` (см. ниже код). Присвойте эти новые имена новому столбцу `df` под названием «Имя в нижнем регистре». Окончательный DataFrame должен выглядеть так:

Name	Email	Lowercase Name
JOHN SMITH	john.smith@gmail.com	john smith
Jane Doe	jdoe@yahoo.com	jane doe
joe schmo	joeschmo@hotmail.com	joe schmo

```
df = pd.DataFrame([  
    ['JOHN SMITH', 'john.smith@gmail.com'],
```

```
['Jane Doe', 'jdoe@yahoo.com'],  
['joe schmo', 'joeschmo@hotmail.com']  
],  
columns=['Name', 'Email'])
```